

INTRODUCCION A SWAGGER

por carlos gallardo ledesma

FLASK

1. como crear una app con flask

Crear una aplicación web básica con Flask implica instalar la biblioteca (pip install flask), importar Flask en un archivo Python, crear una instancia de la aplicación (app = Flask(__name__)), definir una ruta (@app.route('/')) y ejecutarla (app.run()). Flask permite crear webs interactivas rápidamente mediante funciones de Python que retornan texto o HTML

FLASK XD

2. como iniciar la aplicacion

Para iniciar una aplicación en Flask, instala Flask con pip
install flask, crea un archivo app.py, importa Flask e
instancia la app (app = Flask(__name__)). Ejecuta el
servidor de desarrollo usando en la terminal flask run

FLASK

3. que es un template

Un template (plantilla) en Flask es un archivo, generalmente HTML, que contiene la estructura estática de una página web mezclada con marcadores de posición para datos dinámicos. Utiliza el motor Jinja2 para renderizar HTML y se almacenan en una carpeta llamada templates para separar la lógica de negocio (Python) de la interfaz de usuario.

REQUEST

que es

"Request" en inglés significa principalmente solicitud, petición o pedir/solicitar algo formalmente.

Puede funcionar como sustantivo (una petición) o como verbo (solicitar). Se utiliza para expresar la necesidad o el deseo de algo de manera cortés. En contextos técnicos, un request es una petición de datos a un servidor.

REQUEST

tipos

Los tipos de request (solicitudes HTTP) son métodos utilizados por clientes (navegadores) para interactuar con servidores, fundamentales en el desarrollo web y APIs. Los principales son GET (leer), POST (crear), PUT (actualizar total), PATCH (actualizar parcial) y DELETE (eliminar). Otros métodos incluyen HEAD, OPTIONS, CONNECT y TRACE

REQUEST

funcionamiento de

GET: Recupera datos de un recurso. Es seguro (no altera el servidor) e idempotente (resultados iguales siempre).

POST: Envía datos al servidor para crear un nuevo recurso. Se utiliza para enviar formularios o registrar información.

PUT: Actualiza o reemplaza un recurso existente en el servidor. Es idempotente, lo que significa que repetir la solicitud tendrá el mismo efecto que una sola.

DELETE: Elimina un recurso específico identificado por su URL.

SWAGGER



como crear una api

Para crear una API con Flask en Python, instala Flask (`pip install flask`), crea un archivo `app.py`, importa la librería, inicializa `app = Flask(__name__)`, define rutas (`@app.route`) con métodos HTTP (GET, POST), y ejecuta la app con `app.run()`. La API intercambiará datos generalmente en formato JSON.

SWAGGER



como utilizar swagger

Swagger es una herramienta esencial para documentar, diseñar y probar APIs REST automáticamente usando la especificación OpenAPI. Permite visualizar endpoints, probar peticiones (GET, POST, PUT, DELETE) directamente en el navegador y generar documentación interactiva mediante anotaciones en el código o archivos YAML/JSON.

SWAGGER



como corectar flaks con my sql

Conectar Swagger con MySQL no se hace directamente, sino a través de una API (backend) que sirve como intermediario. Swagger documenta y prueba la API, mientras que la API se conecta a MySQL para gestionar los datos.

Aquí tienes los pasos generales y ejemplos según el entorno:

Pasos Generales

Crear la API: Desarrollar un servicio web (Node.js, Spring Boot, .NET, Python) que interactúe con MySQL.

Integrar Swagger/OpenAPI: Agregar las librerías de Swagger al proyecto para documentar los puntos finales (endpoints) de la API.

Configurar conexión a MySQL: Definir en la API las credenciales (host, usuario, contraseña, base de datos).

Generar documentación: Swagger lee el código de la API y genera automáticamente la interfaz UI.